

Llenguatges de Programació

Sessió 3: compiladors



Gerard Escudero i Albert Rubio

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Contingut

- [ANTLR](#)
- Gramàtica
- *Visitor*
- Exercicis
- Funcions
- Tractament d'errors
- Referències

Instal·lació de l'ANTLR4 (per Python)

Requeriments:

- Python 3
- python *pip*

```
sudo apt install python3-pip
```

- python *venv*

```
sudo apt install python3-venv
```

Nota: les versions d'`antlr4` i `antlr4-python3-runtime` han de coincidir.

Windows: s'ha de fer alguna cosa més, seguiu la referència.

- [antlr4-tools reference](#)

Instruccions:

Instal·lació:

```
mkdir treball
cd treball
python3 -m venv lp
source lp/bin/activate
pip install antlr4-tools
antlr4
pip install antlr4-python3-runtime==4.13.
deactivate
```

Ús:

```
mkdir treball
cd treball
source lp/bin/activate
cd practica
make
deactivate
```

Contingut

- ANTLR
- Gramàtica
- *Visitor*
- Exercicis
- Funcions
- Tractament d'errors
- Referències

El primer programa ANTLR

Arxiu de gramàtica `exprs.g4`:

```
// Gramàtica per expressions senzilles

grammar exprs;

root : expr EOF
      ;

expr : expr ( '*' | '/' ) expr
      | expr ( '+' | '-' ) expr
      | NUM
      ;

NUM : [0-9]+ ;
WS  : [ \t\n\r ]+ -> skip ;
```

⚠ Noteu que el nom de l'arxiu ha de concordar amb el de la gramàtica.

- `expr`: definició de la gramàtica per l'aritmètica de nombres naturals.
- `skip`: indica a l'escàner que el token `WS` no ha d'arribar al parser.

Compilació a Python3

La comanda

```
antlr4 -Dlanguage=Python3 -no-listener exprs.g4 # antlr en MacOS
```

genera els arxius:

- `exprsLexer.py` i `exprsLexer.tokens`
- `exprsParser.py` i `exprs.tokens`
- `exprsLexer.interp` i `exprsParser.interp`

⚠ Noteu que els arxius anteriors comencen pel nom de la gramàtica.

Construcció de l'script principal

Script de test `exprs.py`:

```
from antlr4 import *
from exprsLexer import exprsLexer
from exprsParser import exprsParser

input_stream = InputStream(input('? '))
lexer = exprsLexer(input_stream)
token_stream = CommonTokenStream(lexer)
parser = exprsParser(token_stream)
tree = parser.root()
print(tree.toStringTree(recog=parser))
```

Noteu que aquest script processa una única línia d'entrada per consola.

En funcionament:

```
2 + 3 * 4
👉
(root (expr (expr 2) +
            (expr (expr 3) * (expr 4))))
<EOF>
```

```
3 +
👉
line 1:2 missing NUM at '<EOF>'
(root (expr (expr 3) +
            (expr <missing NUM>)) <EOF>)
```

Obtenció de l'entrada

una única línia

```
input_stream = InputStream(input('? '))
```

stdin

```
input_stream = StdinStream()
```

un arxiu passat com a paràmetre

```
input_stream = FileStream(nom_fitxer)
```

arxius amb unicode

```
input_stream = FileStream(nom_fitxer, encoding='utf-8')
```

En aquest cas haurem d'incloure a la gramàtica aquest tipus de caràcters:

```
WORD : [a-zA-Z\u0080-\u00FF]+ ;
```


Notes sobre gramàtiques

Precedència d'operadors:

Amb l'ordre d'escriptura:

```
expr : expr ('*' | '/') expr  
      | expr ('+' | '-') expr  
      | NUM  
      ;
```

Associativitat:

L'associativitat com la potència queda com:

```
expr : <assoc=right> expr '^' expr  
      | INT  
      ;
```

Exercici 1

Afegiu a la gramàtica els operadors de:

- residu de la divisió
- potència
- parèntesis

Tingueu en compte:

- la precedència d'operadors
- l'associativitat a la dreta de la potència

Contingut

- ANTLR
- Gramàtica
- *Visitor*
- Exercicis
- Funcions
- Tractament d'errors
- Referències

Visitors

Els *visitors* són *tree walkers*, un mecanisme per recórrer els ASTs. Amb la comanda:

```
antlr4 -Dlanguage=Python3 -no-listener -visitor exprs.g4 # antlr en MacOS
```

compilarem la gramàtica i generarem la class base del visitador (`exprsVisitor.py`):

```
# Generated from exprs.g4 by ANTLR 4.11.1
from antlr4 import *
if __name__ is not None and "." in __name__:
    from .exprsParser import exprsParser
else:
    from exprsParser import exprsParser

# This class defines a complete generic visitor
class exprsVisitor(ParseTreeVisitor):

    # Visit a parse tree produced by exprsParser
    def visitRoot(self, ctx:exprsParser.RootContext):
        return self.visitChildren(ctx)
    ...
```

`visitRoot` és el *callback* associat a la regla `root` per visitar-la.

Quan hi ha una etiqueta com ara `#suma`, la regla és `visitSuma`.

La crida a `self.visit(node)` visita el visitador associat al tipus de `node`.

Gramàtica amb etiquetes

```
grammar exprs;  
root : expr EOF  
;  
expr : esq=expr op=('*' | '/') dre=expr    # binari  
      | esq=expr op=('+' | '-') dre=expr   # binari  
      | NUM                                # numero  
      ;  
NUM : [0-9]+ ;  
WS  : [ \t\n\r]+ -> skip ;
```

Amb aquesta gramàtica tindrem els mètodes:

- visitRoot, visitBinari i visitNumero

Visitor per avaluar expressions

Classe *visitor* `EvalVisitor` per mostrar l'arbre heretant de la classe base:

```
class EvalVisitor(exprsVisitor):
    def __init__(self):
        self.functions = {'+': lambda x, y: x + y,
                          '-': lambda x, y: x - y,
                          '*': lambda x, y: x * y,
                          '/': lambda x, y: x // y}

    def visitRoot(self, ctx):
        print(self.visit(ctx.expr()))

    def visitBinari(self, ctx):
        expressio1 = ctx.esq
        expressio2 = ctx.dre
        op = ctx.op.text
        return self.functions[op](self.visit(expressio1), self.visit(expressio2))

    def visitNumero(self, ctx):
        return int(ctx.NUM().getText())
```

- `self.visit`: visita un node
- `ctx.esq`: obté l'etiqueta `esq`
- `ctx.expr()`: obté el fill `expr`
- `getText()`: obté el text del node

Ús del visitor

L'script l'hem de modificar:

```
from antlr4 import *
from exprsLexer import exprsLexer
from exprsParser import exprsParser
from exprsVisitor import exprsVisitor

class EvalVisitor(exprsVisitor):
    ...

input_stream = InputStream(input('? '))
lexer = exprsLexer(input_stream)
token_stream = CommonTokenStream(lexer)
parser = exprsParser(token_stream)
tree = parser.root()

visitor = EvalVisitor()
visitor.visit(tree)
```

Exemple:

```
2 + 3 * 5
👉 17
```

Nota: podeu utilitzar més d'un visitor en un script.

Contingut

- ANTLR
- Gramàtica
- *Visitor*
- Exercicis
- Funcions
- Tractament d'errors
- Referències

Exercici 2

Afegiu el tractament d'avaluació per la resta d'operadors de l'exercici 1.

Exercici 3

Definiu una gramàtica i el seu mecanisme d'avaluació/execució per a quelcom tipus:

```
x := 3 + 5  
write x  
y := 3 + x + 5  
write y
```

Nota: es pot utilitzar un diccionari com a taula de símbols.

Exercici 4

Amplieu l'exercici anterior per a que tracti quelcom com el següent:

```
c := 0
b := c + 5
if c = 0 then
  write b
end
```

Exercici 5

Exploreu que passa si realitzem l'exercici anterior sense el token `end`.

Exercici 6

Amplieu l'exercici anterior per a que tracti l'estructura `while`:

```
i := 1
while i <> 11 do
  write i * 2
  i := i + 1
end
```

Contingut

- ANTLR
- Gramàtica
- *Visitor*
- Exercicis
- Funcions
- Tractament d'errors
- Referències

Què passa amb les funcions?

Imagineu una llenguatge tipus:

```
function sm(x, y)
    return x + y
end

main
    a := 1 + 2
    b := a * 2
    write sm(a, b)
end
```

amb només:

- variables locals
- paràmetres per valor

Exercici 7

Amplieu l'exercici anterior per a incloure funcions d'aquest tipus.

Qüestions a tenir en compte:

1. La taula de símbols pot ser una *pila de diccionaris*.
2. En *visitar* la declaració de funcions, per a cada funció, hem de guardar en una estructura:
 - El seu nom (*id*)
 - La seva llista de paràmetres (*ids*)
 - El contexte (node de l'AST) del seu bloc de codi (per a poder fer un `self.visit(bloc)` en trobar la crida)
3. S'ha de gestionar el `return` en cascada.

Exercici 8

Comproveu que el vostre programa funciona amb recursivitat:

```
function fibo(n)
  if n = 0 then
    return 0
  end
  if n = 1 then
    return 1
  end
  return fibo(n-1) + fibo(n-2)
end

main
  a := 1
  while a <> 7 do
    write fibo(a)
    a := a + 1
  end
end
```

Contingut

- ANTLR
- Gramàtica
- *Visitor*
- Exercicis
- Funcions
- Tractament d'errors
- Referències

Errors sintàctics

Antlr té un parell mètodes per tractar-los:

- `getNumberOfSyntaxErrors`: ens indica els errors sintàctics
- `removeErrorListeners`: desactiva els missatges de les excepcions del parser

Exemples:

Amb aquest codi evitem cridar el `visitor` si s'ha produït un error sintàctic.

```
if parser.getNumberOfSyntaxErrors() == 0:  
    visitor = TreeVisitor()  
    visitor.visit(tree)  
else:  
    print(parser.getNumberOfSyntaxErrors(), 'errors de sintaxi.')  
    print(tree.toStringTree(recog=parser))
```

Amb aquest codi evitem els missatges de les excepcions que llença el parser.

```
parser = exprsParser(token_stream)  
parser.removeErrorListeners()  
tree = parser.root()
```

Errors lèxics

Es produeixen quan fem un símbol no reconegut per la gramàtica.

Per evitar aquests errors hem d'afegir una regla al final de la gramàtica:

```
LEXICAL_ERROR : . ;
```

i desactivar els missatges de les excepcions del lexer:

```
lexer = exprsLexer(input_stream)
lexer.removeErrorListeners()
token_stream = CommonTokenStream(lexer)
```


Contingut

- ANTLR
- Gramàtica
- *Visitor*
- Exercicis
- Funcions
- Tractament d'errors
- Referències

Referències

1. Terence Parr. *The Definitive ANTLR 4 Reference*, 2nd Edition. Pragmatic Bookshelf, 2013.
2. Alan Hohn. *ANTLR4 Python Example*. Últim accés: 26/1/2019.
<https://github.com/AlanHohn/antlr4-python>
3. Guillem Godoy i Ramón Ferrer. *Parsing and AST construction with PCCTS*. Materials d'LP, 2011.